

Cheat Sheet – Programmazione ed Algoritmica

4ª Prova in Itinere · Diego Stefanini

Programmazione Dinamica

Quando si applica

Due condizioni devono valere contemporaneamente:

- **Sovrapposizione dei sotto-problemi:** la ricorsione naturale ricalcola più volte gli stessi sotto-problemi.
- **Sottostruttura ottima:** la soluzione ottima del problema si costruisce da soluzioni ottime di sotto-problemi.

PD vs Divide et Impera: D&I suddivide in sotto-problemi **indipendenti**; PD sfrutta invece la **sovrapposizione** memorizzando i risultati.

PD vs Greedy: Greedy fa scelte localmente ottime senza tornare indietro; PD esplora tutte le scelte tramite la ricorrenza e sceglie la migliore.

Due approcci equivalenti

- **Top-down (memoization):** ricorsione + tabella di cache. Scrivi direttamente la ricorrenza, ricordando ciò che hai già calcolato.
- **Bottom-up (tabulation):** ciclo iterativo che riempie la tabella nell'ordine giusto (dai casi base verso il problema completo). Di solito più efficiente in pratica.

Schema di risoluzione (6 passi)

1. **Definisci DP[...]:** cosa rappresenta ciascuna cella (a parole, non solo formule).
2. **Ricorrenza:** esprimi DP[...] in funzione di celle "più piccole".
3. **Casi base:** celle risolvibili direttamente.
4. **Ordine di riempimento:** tale che quando calcoli una cella i suoi dipendenti sono già pronti.
5. **Estrai la risposta** dalla cella finale.
6. **Ricostruzione (opz):** tabella ausiliaria pred[...] che registra la scelta vincente.

Trucco mentale: chiediti «qual è l'ultima scelta/decisione?». Da lì discendono i sotto-problemi e la ricorrenza diventa un caso "prendo / non prendo l'i-esimo" oppure "scelgo la sorgente i del sotto-problema".

Tabella riepilogativa dei problemi PD

Problema	Forza bruta	PD	Tabella
Fibonacci	$O(\varphi^n)$	$\Theta(n)$	$1D, n+1$
Taglio Corda	$O(2^n)$	$\Theta(n^2)$	$1D, n+1$
LCS	$O(n \cdot 2^m)$	$\Theta(mn)$	$(m+1) \times (n+1)$
LIS	$O(2^n)$	$\Theta(n^2)$	$1D, n$
Partition	$O(n2^n)$	$O(ns)$	$(n+1) \times (s+1)$
Knapsack 0/1	$O(n2^n)$	$O(nW)$	$(n+1) \times (W+1)$
Cammino griglia	$O(2^{2n})$	$\Theta(n^2)$	$n \times n$

Taglio della Corda (Rod Cutting) – $\Theta(n^2)$

Asta di lunghezza n , prezzi $p[1..n]$ per pezzi interi. Massimizzare il ricavo.

Ricorrenza: $r[0] = 0$, $r[j] = \max_{1 \leq i \leq j} (p[i] + r[j-i])$.

```
extendedBottomUpCutRod(p, n):
    r[0] = 0
    for j = 1 to n:
        q = -infinity
        for i = 1 to j:
            if q < p[i] + r[j-i]:
                q = p[i] + r[j-i]; s[j] = i
        r[j] = q
    return r, s
```

Ricostruzione: stampa $s[n]$, poi passa a $n - s[n]$, ripeti finché resto = 0.

```
printCutRod(p, n):
    (r, s) = extendedBottomUpCutRod(p, n)
    while n > 0:
        print s[n]
        n = n - s[n]
```

Sottostruttura ottima (per la dimostrazione): se $r[n]$ è ottimo e include un primo taglio di lunghezza i^* , allora $r[n - i^*]$ deve essere anch'esso ottimo (altrimenti potrei migliorare). **Tempo:** $\Theta(n^2)$, **spazio:** $\Theta(n)$.

LCS (Sottosequenza Comune Più Lunga) – $\Theta(mn)$

Date $X[1..m]$ e $Y[1..n]$, trovare la più lunga sequenza Z sottosequenza di entrambe.

Ricorrenza: $c[i, j]$ = lunghezza LCS di $X[1..i]$ e $Y[1..j]$.

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i-1, j-1] + 1 & \text{se } X[i] = Y[j] \\ \max(c[i-1, j], c[i, j-1]) & \text{altrimenti} \end{cases}$$

```
lcsLength(X, Y):
    m = X.length; n = Y.length
    for i = 0 to m: c[i][0] = 0
    for j = 0 to n: c[0][j] = 0
    for i = 1 to m:
        for j = 1 to n:
            if X[i] == Y[j]:
                c[i][j] = c[i-1][j-1] + 1
                b[i][j] = "DIAG"
            elif c[i-1][j] >= c[i][j-1]:
                c[i][j] = c[i-1][j]; b[i][j] = "UP"
            else:
                c[i][j] = c[i][j-1]; b[i][j] = "LEFT"
    return c, b
```

Ricostruzione (a ritroso da (m, n)): printLCS(b, X, i, j):

```
if i == 0 || j == 0: return
if b[i][j] == "DIAG":
    printLCS(b, X, i-1, j-1); print X[i]
elif b[i][j] == "UP":
```

```
    printLCS(b, X, i-1, j)
else:
    printLCS(b, X, i, j-1)
```

Esempio (RISTOTTO/RISTORO): tabella 9×8 , $c[8, 7] = 6$, una LCS è RISTOO. **Tempo:** $\Theta(mn)$, **spazio:** $\Theta(mn)$ (riducibile a $\Theta(\min(m, n))$) se non serve ricostruire).

Cammini su Griglia – Conteggio

Scacchiera $n \times n$, mosse giù o destra, contare i cammini da $(0, 0)$ a $(n-1, n-1)$.

Ricorrenza: $DP[i][j] = DP[i-1][j] + DP[i][j-1]$. **Casi base:** $DP[0][j] = 1$, $DP[i][0] = 1$.

```
countPaths(n):
    for j = 0 to n-1: DP[0][j] = 1
    for i = 0 to n-1: DP[i][0] = 1
    for i = 1 to n-1:
        for j = 1 to n-1:
            DP[i][j] = DP[i-1][j] + DP[i][j-1]
    return DP[n-1][n-1]
```

Tempo e spazio: $\Theta(n^2)$. Formula chiusa: $\binom{2n-2}{n-1}$.

Cammini su Griglia – Valore Massimo

Stessa griglia, valori $c[i][j]$. Trovare il cammino con somma massima.

Ricorrenza: $DP[i][j] = c[i][j] + \max(DP[i-1][j], DP[i][j-1])$.

```
maxPath(C, n):
    DP[0][0] = C[0][0]
    for j = 1 to n-1: DP[0][j] = DP[0][j-1] + C[0][j]
    for i = 1 to n-1: DP[i][0] = DP[i-1][0] + C[i][0]
    for i = 1 to n-1:
        for j = 1 to n-1:
            DP[i][j] = C[i][j] + max(DP[i-1][j], DP[i][j-1])
    return DP[n-1][n-1]
```

Cammini su Griglia – Costo Minimo

Variante: ogni casella ha un costo $C[i][j]$, minimizzare la somma. Identica all'algoritmo precedente sostituendo max con min.

Ricorrenza: $DP[i][j] = c[i][j] + \min(DP[i-1][j], DP[i][j-1])$.

LIS (Longest Increasing Subsequence) – $O(n^2)$

Array $L[0..n-1]$ di interi distinti. Più lunga sottosequenza strettamente crescente.

Ricorrenza (definizione "termina in i "): $LIS[i] = 1 + \max\{LIS[j] : j < i, L[j] < L[i]\}$, oppure 1 se nessun j soddisfa la condizione.

```
lisLength(L, n):
    for i = 0 to n-1:
        LIS[i] = 1; pred[i] = -1
        for j = 0 to i-1:
            if L[j] < L[i] && LIS[j] + 1 > LIS[i]:
                LIS[i] = LIS[j] + 1; pred[i] = j
    return max(LIS[0..n-1])
```

Ricostruzione: trova i^* che massimizza $LIS[i]$; segui pred[i^*] a ritroso e inverti. **Tempo:** $\Theta(n^2)$, **spazio:** $\Theta(n)$.

Partition (Subset Sum) – $O(ns)$

Dato $A = \{a_1, \dots, a_n\}$ di interi positivi con somma $2s$, esiste $A' \subseteq A$ con somma s ?

Ricorrenza: $p[i, j]$ = vero sse esiste sottoinsieme dei primi i elementi con somma j .

$$p[i, j] = \begin{cases} \text{true} & \text{se } j = 0 \\ \text{false} & \text{se } i = 0 \text{ e } j > 0 \\ p[i-1, j] & \text{se } a_i > j \\ p[i-1, j] \vee p[i-1, j-a_i] & \text{altrimenti} \end{cases}$$

Lettura: «posso ottenere somma j usando solo i primi i elementi?»

Risposta finale in $p[n, s]$.

```
partition(A, n):
    totale = sum(A)
    if totale is odd: return false
    s = totale / 2
    for i = 0 to n: p[i][0] = true
    for j = 1 to s: p[0][j] = false
    for i = 1 to n:
        for j = 1 to s:
            if A[i] > j: p[i][j] = p[i-1][j]
            else: p[i][j] = p[i-1][j] OR p[i-1][j - A[i]]
    return p[n][s]
```

Ricostruzione del sottoinsieme (a ritroso da $p[n, s]$): subset(A, p, n, s):

```
if not p[n][s]: print 'no'; return
i = n; j = s
while i > 0:
    if p[i-1][j] == false: // ho preso a_i
        print A[i]; j = j - A[i]
    i = i - 1
```

Pseudo-polinomiale: $O(ns)$ è polinomiale in n e nel valore di s , ma esponenziale nel numero di bit per rappresentarlo. Partition è NP-completo: se s cresce esponenzialmente nei bit dell'input, l'algoritmo non è trattabile.

Knapsack 0/1 – $O(nW)$

n oggetti con peso w_i e valore v_i . Zaino di capacità W . Massimizzare il valore totale senza superare W (ogni oggetto si prende intero o non si prende).

Ricorrenza: $c[i, w]$ = massimo valore usando solo i primi i oggetti con peso al più w .

$$c[i, w] = \begin{cases} 0 & \text{se } i = 0 \text{ o } w = 0 \\ c[i-1, w] & \text{se } w_i > w \\ \max(c[i-1, w], v_i + c[i-1, w-w_i]) & \text{altrimenti} \end{cases}$$

```
knapsack01(v, w, n, W):
    for j = 0 to W: c[0][j] = 0
    for i = 0 to n: c[i][0] = 0
    for i = 1 to n:
```

```
for j = 1 to W:
  if w[i] > j:
    c[i][j] = c[i-1][j]
  else:
    c[i][j] = max(c[i-1][j], v[i] + c[i-1][j - w[i]])
return c[n][W]
```

Ricostruzione: parti da $c[n, W]$. Se $c[i, w] = c[i - 1, w]$ allora oggetto i non preso; altrimenti preso e passa a $c[i - 1, w - w_i]$. Tempo $O(n)$.

Esempio: $a = (1, 60), b = (2, 100), c = (3, 120), W = 5$. Soluzione ottima: $\{b, c\}$ con peso 5 e valore 220.

Min/Max Numero di Oggetti per Somma Esatta
Pattern (souvenir): dato $V[1..n]$ e capacità C , trovare il minimo numero di oggetti la cui somma sia esattamente C .
Ricorrenza: $m[i, j] = \min$ numero di oggetti tra i primi i con somma esatta $j, +\infty$ se impossibile.

$$m[i, j] = \begin{cases} 0 & \text{se } j = 0 \\ +\infty & \text{se } i = 0 \text{ e } j > 0 \\ m[i - 1, j] & \text{se } V[i] > j \\ \min(m[i - 1, j], 1 + m[i - 1, j - V[i]]) & \text{altrimenti} \end{cases}$$

Risposta: $m[n][C]$ (se $+\infty$ non esiste). **Tempo:** $O(nC)$.
Variante “carrello/ascensore” (max numero di oggetti con somma $\leq K$): identico ma si massimizza il conteggio invece di minimizzare. Ricorrenza simmetrica con max e caso base 0.

- Checklist d'esame (PD)**
1. Enuncia DP[...] a parole (non solo formule).
 2. Scrivi la ricorrenza con tutti i casi (inclusi quelli di frontiera).
 3. Specifica esplicitamente i casi base.
 4. Giustifica l'ordine dei cicli.
 5. Mostra la tabella su un esempio piccolo.
 6. Analizza tempo e spazio.
 7. Per la soluzione ottima (non solo il valore), mostra pred e ricostruzione.

Grafi – Definizioni

Concetti base

Grafo $G = (V, E), n = |V|, m = |E|$.
• **Non orientato:** archi $\{u, v\}, 0 \leq m \leq \binom{n}{2} = \frac{n(n-1)}{2}$.
• **Orientato:** archi $(u, v) \neq (v, u), 0 \leq m \leq n(n-1)$.
• **Sparso:** $m = O(n)$. **Denso:** $m = \Theta(n^2)$.
• K_n (grafo completo): $m = \binom{n}{2}$, ogni coppia è connessa.

Grado: per non orientato $\delta(v)$ = archi incidenti su $v, \sum_v \delta(v) = 2m$. Per orientato: $\delta_e(v)$ entrante, $\delta_u(v)$ uscente, $\sum \delta_e = \sum \delta_u = m$.
Cammino $\langle x_0, \dots, x_k \rangle: (x_{i-1}, x_i) \in E$. **Lunghezza** = k (numero di archi).
Semplice = vertici distinti. **Ciclo** = $x_k = x_0$. **Distanza** $\delta(u, v)$ = lunghezza del cammino minimo (o $+\infty$).

Connesso (non orientato): ogni coppia ha un cammino. **Componente connessa:** sotto-grafo connesso massimale. **Fortemente connesso** (orientato): per ogni u, v esiste $u \rightsquigarrow v$ e $v \rightsquigarrow u$.

Aciclico: nessun ciclo. **Albero (libero):** non orientato, connesso, aciclico – equivalente a $|E| = |V| - 1$ + connesso. **Foresta:** unione di alberi. **DAG:** orientato aciclico.

Matrice vs Liste di adiacenza

Operazione	Matrice	Liste
Spazio	$\Theta(n^2)$	$\Theta(n + m)$
adiacenti(u,v)	$O(1)$	$O(\delta(u))$
grado(u)	$\Theta(n)$	$\Theta(\delta(u))$
aggiungiArco	$O(1)$	$O(1)$
rimuoviArco	$O(1)$	$O(\delta(u) + \delta(v))$
Scorrere Adj[u]	$\Theta(n)$	$\Theta(\delta(u))$

Regola pratica: matrice se grafo denso o se serve adiacenti in $O(1)$; liste se sparso o se serve scorrere efficientemente i vicini.

BFS – Visita in Ampiezza

Idea e codice

Coda FIFO. Scopre i vertici in ordine di **distanza crescente** dalla sorgente. Per ogni v : $v.d$ (distanza), $v.\pi$ (predecessore), $v.\text{color} \in \{B, G, N\}$.

```
BFS(G, s):
  for v in V \ {s}:
    v.color = B; v.d = +inf; v.pi = NIL
  s.color = G; s.d = 0; s.pi = NIL
  Q = coda vuota; enqueue(Q, s)
  while Q non vuota:
    u = dequeue(Q)
    for v in Adj[u]:
      if v.color == B:
        v.color = G; v.d = u.d + 1; v.pi = u
        enqueue(Q, v)
    u.color = N
```

Complessità: $T(n, m) = O(n + m)$. **Correttezza:** $v.d = \delta(s, v)$ per ogni v raggiungibile.

```
PRINT-PATH(G, s, v):
  if v == s: print s
  elif v.pi == NIL: print 'no path'
  else: PRINT-PATH(G, s, v.pi); print v
```

Albero BFS

Sotto-grafo $G_\pi = (V_\pi, E_\pi), V_\pi = \{v : v.\pi \neq \text{NIL}\} \cup \{s\}, E_\pi = \{(v.\pi, v)\}$. Contiene **tutti** i cammini minimi da s .

Proprietà chiave

- **Triangolare:** $\delta(s, v) \leq \delta(s, u) + 1$ per ogni arco (u, v) .
- **Coda monotona:** in Q ci sono al più 2 valori distinti di d , ovvero k e $k + 1$.
- **Limite superiore:** sempre $v.d \geq \delta(s, v)$.

DFS – Visita in Profondità

Idea e codice

Esplora “il più lontano possibile prima di tornare indietro”. Visita **tutto** il grafo (non solo una componente), produce una **foresta DF** e assegna due timestamp $v.d$ (scoperta) e $v.f$ (fine), con $v.d < v.f$ in $[1, 2|V|]$.

```
DFS(G):
  for u in V:
    u.color = B; u.pi = NIL
    time = 0
  for u in V:
    if u.color == B:
      DFS-VISIT(G, u)
```

```
DFS-VISIT(G, u):
  time = time + 1; u.d = time
  u.color = G
  for v in Adj[u]:
    if v.color == B:
      v.pi = u
      DFS-VISIT(G, v)
  u.color = N
  time = time + 1; u.f = time
```

Complessità: $T(n, m) = \Theta(n + m)$.

Classificazione degli archi

Per ogni arco (u, v) , al momento dell'ispezione (colore di v):

- **Arco d'albero** – v Bianco: $(u, v) \in E_\pi$.
- **Arco all'indietro** – v Grigio: v è antenato di u .
- **Arco in avanti** – v Nero, $u.d < v.d$: v è discendente non diretto.
- **Arco trasversale** – v Nero, $u.d > v.d$: nessuna relazione di discendenza.

Nei grafi non orientati esistono solo archi d'albero o all'indietro.

Teoremi fondamentali

Parentesi: per ogni u, v : o gli intervalli $[u.d, u.f]$ e $[v.d, v.f]$ sono disgiunti, o uno contiene l'altro (relazione di antenato/discendente).
Cammino bianco: v è discendente di u nella foresta DF $\leftrightarrow$ al tempo $u.d$ esiste un cammino $u \rightsquigarrow v$ fatto solo di vertici bianchi.
Aciclicità: un grafo orientato è aciclico $\leftrightarrow$ una DFS non genera archi all'indietro.

Cammini Minimi e Dijkstra

Grafi pesati

Funzione peso $\omega : E \rightarrow \mathbb{R}$. Peso di un cammino $p = \langle v_0, \dots, v_k \rangle$:

$$\omega(p) = \sum_{i=1}^k \omega(v_{i-1}, v_i)$$

$\delta(u, v) = \min\{\omega(p) : u \rightsquigarrow v\}$, oppure $+\infty$.
Sottostruttura ottima: ogni sotto-cammino di un cammino minimo è a sua volta minimo (i cammini minimi sono semplici).
Disuguaglianza triangolare pesata: $\delta(s, v) \leq \delta(s, u) + \omega(u, v)$ per ogni arco (u, v) .

Inizializzazione e rilassamento

Per ogni vertice: $v.d$ = stima (limite superiore di $\delta(s, v)$, parte da $+\infty$, cala monotonamente), $v.\pi$ = predecessore.

```
INITIALIZE-SINGLE-SOURCE(G, s):
  for v in V: v.d = +inf; v.pi = NIL
  s.d = 0
```

```
RELAX(u, v, w):
  if v.d > u.d + w(u, v):
    v.d = u.d + w(u, v); v.pi = u
```

Algoritmo di Dijkstra

Ipotesi: $\omega(u, v) \geq 0$ per ogni arco. Non funziona con pesi negativi.

```
DIJKSTRA(G, w, s):
  INITIALIZE-SINGLE-SOURCE(G, s)
  PQ = nuova coda di MIN-priorita
  for v in V: INSERT(PQ, v) // priorit  : v.d
  while PQ != vuota:
    u = EXTRACT-MIN(PQ)
    for v in Adj[u]:
      if v.d > u.d + w(u, v):
        v.d = u.d + w(u, v); v.pi = u
        DECREASE-KEY(PQ, v, v.d)
```

Idea: greedy che estrae sempre il vertice con stima minima (   gi   sicuro che sia $\delta(s, u)$) e rilassa gli archi uscenti.

Complessit   di Dijkstra

$T = O(V \cdot \text{extractMin} + E \cdot \text{decreaseKey})$			
PQ	EXT-MIN	DEC-KEY	Totale
Array indicizzato	$O(V)$	$O(1)$	$O(V ^2)$
MIN-heap binario	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$

Array per grafi densi ($m = \Theta(n^2)$). **Heap binario** per grafi sparsi ($m = O(n)$).

Cammini minimi: varianti del problema

- **Sorgente unica s :** da s a tutti gli altri (Dijkstra).
- **Destinazione unica t :** invertendo gli archi si riconduce al precedente.
- **Coppia (u, v) :** asintoticamente costa quanto la sorgente unica.

- **Tutte le coppie:** iterare su ogni sorgente (esistono algoritmi ad hoc come Floyd-Warshall, fuori programma).
- Pesi negativi:** Dijkstra non funziona; serve Bellman-Ford (fuori programma). Cicli di peso negativo $\rightarrow \delta = -\infty$.
- Grafi non pesati:** BFS calcola δ in $O(n + m)$.

Problem Solving su Grafi

Mappa: quale visita usare?

- **Connessione, raggiungibilità, distanze non pesate, livelli:** BFS.
- **Aciclicità, ordinamento topologico, classificazione archi, struttura ad albero/cicli:** DFS.
- **Distanze pesate (peso ≥ 0):** Dijkstra.
- **Bipartizione, distanza esatta, “tutti i nodi a distanza k ”:** BFS (uso il livello).

Connessione (non orientato)

Una sola BFS da un vertice qualsiasi: se al termine qualche vertice resta bianco, G non è connesso.

```
connesso(G):
    s = qualsiasi vertice in V
    BFS(G, s)
    for v in V:
        if v.color == B: return false
    return true Tempo:  $\Theta(n + m)$ .
```

Conta Componenti Connesse

BFS (o DFS) su tutto il grafo, riavviando da ogni vertice bianco; ogni nuovo avvio = nuova componente.

```
contaCC(G):
    for v in V: v.color = B
    cc = 0
    for v in V:
        if v.color == B:
            cc = cc + 1
            BFS-modificata(G, v) // colora la componente
    return cc Tempo:  $\Theta(n + m)$ .
```

Aciclicità

- Non orientato:** DFS; G ciclico $\leftrightarrow$ esiste un arco all’indietro. In alternativa, se G è connesso: ciclico $\leftrightarrow |E| > |V| - 1$. **Tempo:** $\Theta(n + m)$.
- Orientato:** DFS; G aciclico $\leftrightarrow$ DFS non genera archi all’indietro. Si rileva con i colori: arco (u, v) con v Grigio. **Tempo:** $\Theta(n + m)$.

```
aciclico(G): // orientato
    DFS(G, monitorando archi:
        durante DFS-VISIT(u), per ogni v in Adj[u]:
            if v.color == G: return false // back edge
    return true
```

È un Albero?

G non orientato è albero $\leftrightarrow$ è connesso e $|E| = |V| - 1$ (oppure connesso e aciclico).

```
isTree(G):
    if |E| != |V| - 1: return false
    return connesso(G)
```

Min Archi per Rendere Connesso

Se G ha k componenti connesse, servono esattamente $k - 1$ archi per renderlo connesso.

```
minArchi(G):
    return contaCC(G) - 1
```

Ordinamento Topologico (DAG)

Definizione: ordinamento lineare dei vertici di un DAG tale che per ogni arco (u, v) , u precede v .

Algoritmo: DFS, poi inseriamo ogni vertice in testa a una lista al momento di colorarlo Nero (cioè in ordine decrescente di f).

```
topSort(G):
    L = lista vuota
    DFS(G, e nella riga 'u.color = N' aggiungi:
        inserisci u in testa a L
    return L
```

Esiste un ordinamento topologico $\leftrightarrow G$ è un DAG. Se G ha cicli, prima rimuovi un arco di un ciclo (ad es. arco all’indietro). **Tempo:** $\Theta(n + m)$.

Super-Pozzo (Orientato, Matrice di Adiacenza)

Definizione: vertice con grado uscente 0 e grado entrante $n - 1$. Se esiste, è unico.

Idea $O(n)$: due indici i, j . Se $A[i, j] = 1$ allora i ha un arco uscente $\rightarrow i$ non è pozzo $\rightarrow$ avanza i . Se $A[i, j] = 0$ allora j non riceve da $i \rightarrow j$ non è pozzo $\rightarrow$ avanza j . Alla fine si ha un solo candidato; verifica esplicita in $O(n)$.

```
superPozzo(A, n):
    i = 1; j = 2
    while j <= n:
        if A[i][j] == 1: i = j; j = j + 1
        else: j = j + 1
    // i e l'unico candidato
    for k = 1 to n:
        if k != i:
            if A[i][k] != 0 || A[k][i] != 1:
                return -1
    return i Tempo:  $\Theta(n)$  con matrice di adiacenza.
```

Y sta su un cammino X $\rightarrow$ Z?

y è su un cammino $x \rightsquigarrow z \leftrightarrow y$ è raggiungibile da x e z è raggiungibile da y .

```
onPath(G, x, y, z):
    raggiungibili_x = BFS(G, x)
    raggiungibili_y = BFS(G, y)
    return (y in raggiungibili_x) AND
        (z in raggiungibili_y) Tempo:  $O(n + m)$ .
```

Numero di Vertici a Distanza Massima da r

BFS da r , poi conta i vertici con $v.d$ uguale al massimo.

```
contaDistMax(G, r):
    BFS(G, r)
    M = max(v.d : v.d != +inf)
    k = 0
    for v in V:
        if v.d == M: k = k + 1
    return k Tempo:  $O(n + m)$ .
```

Rete Valida (Multi-Sorgente BFS)

Ogni vertice ha tipo transmitter, receiver o switch. La rete è valida se ogni receiver è raggiungibile da almeno un transmitter.

Idea: BFS multi-sorgente partendo da tutti i transmitter contemporaneamente.

```
reteValida(G):
    for v in V: v.color = B; v.d = +inf
    Q = coda vuota
    for v in V:
        if v.type == transmitter:
            v.color = G; v.d = 0; enqueue(Q, v)
    while Q non vuota:
        u = dequeue(Q)
        for v in Adj[u]:
            if v.color == B:
                v.color = G; v.d = u.d + 1
                enqueue(Q, v)
        u.color = N
    for v in V:
        if v.type == receiver && v.color == B:
            return false
    return true Tempo:  $O(n + m)$ .
```

Conta Nodi Raggiungibili da s e t alla Stessa Distanza

Due BFS indipendenti. Per ogni vertice v , controllo se $v.d_s$ e $v.d_t$ sono entrambi finiti e uguali.

```
contaStessaDist(G, s, t):
    BFS(G, s); salva v.d in d_s[v]
    BFS(G, t); salva v.d in d_t[v]
    k = 0
    for v in V:
        if d_s[v] != +inf && d_t[v] != +inf
            && d_s[v] == d_t[v]:
                k = k + 1
    return k Tempo:  $O(n + m)$ .
```

Cammino con Tutti i Vertici di Chiave Pari

Variante di BFS con vincolo di accettazione: si esplorano solo i vertici che soddisfano la condizione.

```
camminoPari(G, x, y):
    if x.chiave dispari || y.chiave dispari:
        return false
    for v in V: v.color = B
    Q = vuota; x.color = G; enqueue(Q, x)
    while Q non vuota:
        u = dequeue(Q)
        if u == y: return true
        for v in Adj[u]:
            if v.color == B && v.chiave pari:
                v.color = G; enqueue(Q, v)
        u.color = N
    return false Tempo:  $O(n + m)$ .
```

Pattern: Adattare BFS/DFS

Cosa cambia tipicamente

- La struttura di BFS (coda, colori) o DFS (ricorsione, timestamp) resta. Si modifica:
1. **Cosa si accoda** (singolo vertice $\rightarrow$ molteplici sorgenti).
 2. **Cosa si filtra** nell’inserimento (vincoli sulle chiavi, sui pesi, sul tipo).
 3. **Cosa si raccoglie** (lista raggiungibili, contatori, somme, distanze massime).
 4. **Cosa si fa al ritorno** dalla DFS-VISIT (post-ordine, ordinamento topologico).
 5. **Quando ci si ferma** (target raggiunto, condizione soddisfatta).

BFS che restituisce i raggiungibili

```
BFS-reach(G, s):
    R = lista vuota
    for v in V: v.color = B
    s.color = G; enqueue(Q, s)
    while Q non vuota:
        u = dequeue(Q); append R, u
        for v in Adj[u]:
            if v.color == B:
                v.color = G; enqueue(Q, v)
        u.color = N
    return R
```

BFS Multi-Sorgente

Coda inizializzata con tutte le sorgenti $S \subseteq V$. Il risultato è la distanza al più vicino membro di S .

```
BFS-multi(G, S):
    for v in V: v.color = B; v.d = +inf
    Q = vuota
    for s in S:
        s.color = G; s.d = 0; enqueue(Q, s)
    // resto invariato
```

Applicazioni: rete valida (sorgenti = transmitter), incendi che si propagano, distanza dal “più vicino tra i k ”.

BFS su Grafo Sconosciuto a Priori (con Dizionario)
Quando V non è noto (es. crawler web), si usa un dizionario D (tabella hash) per tener traccia dei vertici già visitati. Il colore Bianco diventa “non in D ”.

```
BFS-dict(s):
    D = dizionario vuoto; D[s] = (G, 0, NIL)
    Q = vuota; enqueue(Q, s)
    while Q non vuota:
        u = dequeue(Q)
        for v in vicini(u):      // chiamata API
            if v not in D:
                D[v] = (G, D[u].d + 1, u)
                enqueue(Q, v)
    D[u].color = N
```

Costo medio: $\Theta(n_s + m_s)$ con tabella hash, dove n_s, m_s sono il numero di vertici/archi raggiungibili da s .

BFS con Stop Anticipato
Cerco il primo vertice che soddisfa una condizione: appena lo trovo, esco. Mantiene complessità $O(n + m)$ nel caso pessimo, ma in pratica spesso molto meno.

```
BFS-find(G, s, ok):
    // ok(v) è una funzione booleana
    ... codice BFS ...
    when (v scoperto):
        if ok(v): return v
```

DFS con Restituzione (post-ordine)
Tipico per ordinamento topologico, oppure per calcoli ricorsivi su alberi DF.

```
DFS-VISIT-collect(G, u, L):
    u.color = G; u.d = ++time
    for v in Adj[u]:
        if v.color == B:
            v.pi = u; DFS-VISIT-collect(G, v, L)
    u.color = N; u.f = ++time
    prepend(L, u)      // post-ordine inverso
```

Conversione Liste → Matrice
listToMat(G):
A = matrice n x n a 0
for u in V:
 for v in Adj[u]: A[u][v] = 1
return A **Tempo:** $\Theta(n^2 + m) = \Theta(n^2)$.

Problem Solving su Array

Pattern ricorrenti

- **Ricerca binaria:** array ordinato, $O(\log n)$. Buona quando esiste un **predicato monotono**.
- **Punto di transizione:** trovare il primo indice i con $A[i] \geq$ soglia in array ordinato — binaria.
- **Due puntatori (sliding window):** somma/prodotto di sotto-array con vincoli — $O(n)$.
- **Prefix sum:** per query “somma di $A[l..r]$ ” in $O(1)$ dopo un pre-calcolo $O(n)$.
- **Conta inversioni / coppie speciali:** tipicamente $O(n \log n)$ via merge-sort modificato.
- **Massimo subarray:** algoritmo di Kadane in $O(n)$ — variante PD: $f[i] = \max(A[i], f[i - 1] + A[i])$.

Massimo Subarray (Kadane) — $O(n)$

```
kadane(A, n):
    best = A[0]; cur = A[0]
    for i = 1 to n-1:
        cur = max(A[i], cur + A[i])
        best = max(best, cur)
    return best
```

Divide et Impera su Array
Ogni volta che la dimensione si dimezza e il “lavoro extra” è polinomiale, applica il **Master Theorem**: $T(n) = aT(\frac{n}{b}) + \Theta(n^c \log^k n)$.

Problem Solving su Alberi

Schemi ricorsivi tipici

Su un albero binario, la maggior parte dei problemi si risolve con ricor-sione **post-ordine** (calcola sui figli, poi combina al nodo).
$$T(u) = \text{combina}(T(u.\text{left}), T(u.\text{right}), u)$$

```
template(u):
    if u == NIL: return CASO_BASE
    L = template(u.left)
    R = template(u.right)
    return combina(L, R, u)
```

Esempi di combinazione

Problema	Caso base	Combinazione
Numero nodi	0	$L + R + 1$
Altezza	-1	$1 + \max(L, R)$
Somma chiavi	0	$L + R + u.\text{key}$
È bilanciato?	true	bilanciati e $ h_L - h_R \leq 1$
Diametro	(0, -1)	vedi sotto
È BST?	true	check ordini

Diametro (lunghezza max cammino tra due foglie): in ogni nodo u , il candidato è $h_L + h_R + 2$ (cammino che passa per u). Si tiene il massimo globale durante la ricorsione. $O(n)$.

BST — Operazioni in $O(h)$
Ricerca, insert, delete in $O(h)$ con h = altezza. Nel caso pessimo (sbilan-ciato) $h = \Theta(n)$; bilanciato $h = \Theta(\log n)$.

Visita in-order di un BST → chiavi in ordine crescente.

Checklist d’Esame

Prima di rispondere

1. **Leggi attentamente:** dimensioni, vincoli, cosa è dato (matrice/liste? pesi?), cosa si chiede (esistenza? conteggio? cammino?).
2. **Identifica la categoria:** ricerca/scelta esponenziale → PD; raggiungi-bilità/distanze → visite; pesi non negativi → Dijkstra.
3. **Controlla rappresentazione:** matrice o liste? La complessità target dipende da questo.
4. **Riusa algoritmi noti:** ogni esercizio è quasi sempre un **adattamento**, non un’invenzione.

Quando scrivi la soluzione

1. Definisci tipi di dato e attributi che usi.
2. Scrivi pseudocodice ordinato (indentazione).
3. Spiega l’intuizione **prima** del codice.
4. Calcola la complessità giustificando ogni ciclo / ogni operazione costosa.
5. Mostra un esempio piccolo se il tempo lo permette.
6. Sulla PD: **casi base + ricorrenza + ordine di riempimento + ricostruzione**.
7. Sui grafi: dichiara la rappresentazione attesa, scegli BFS/DFS/Dijk-stra coerentemente.

Errori da evitare

- Confondere $|V|$ con $|E|$ nella complessità (sparsi vs densi).
- Usare Dijkstra con pesi negativi.
- Dimenticare l’inizializzazione dei colori in BFS/DFS.
- Ricostruzione PD: confondere “preso” con “non preso” leggendo la tabella.
- Scrivere ricorrenze senza casi base.
- Affermare $T = \Theta(n + m)$ quando si sta usando una matrice di adia-cenza ($O(n^2)$).